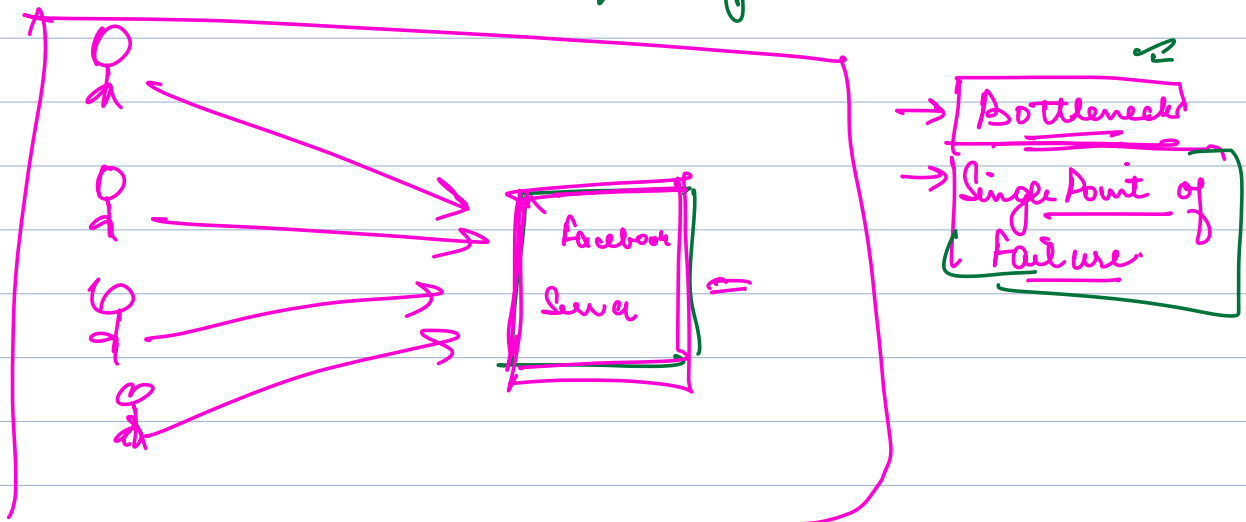
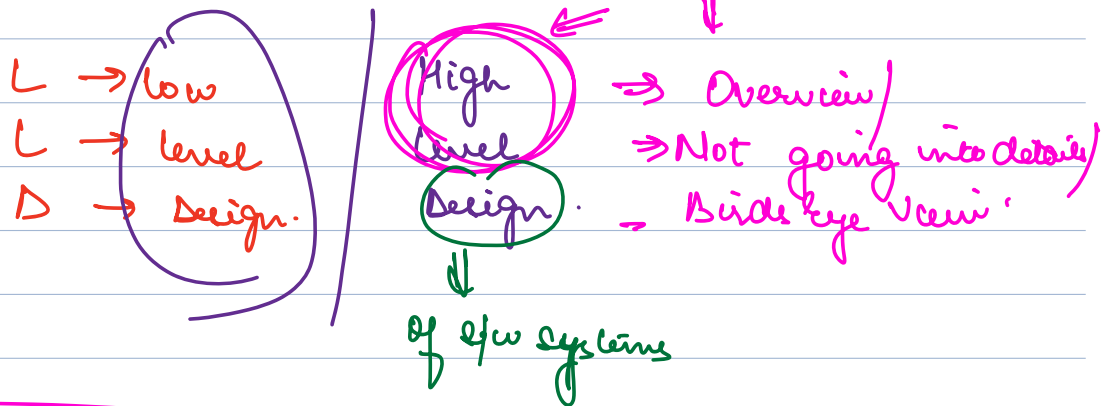


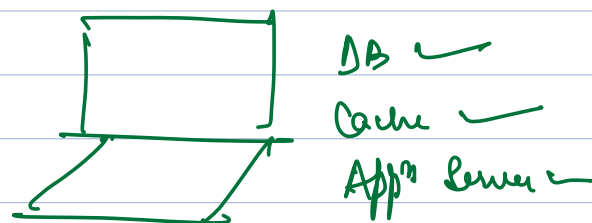
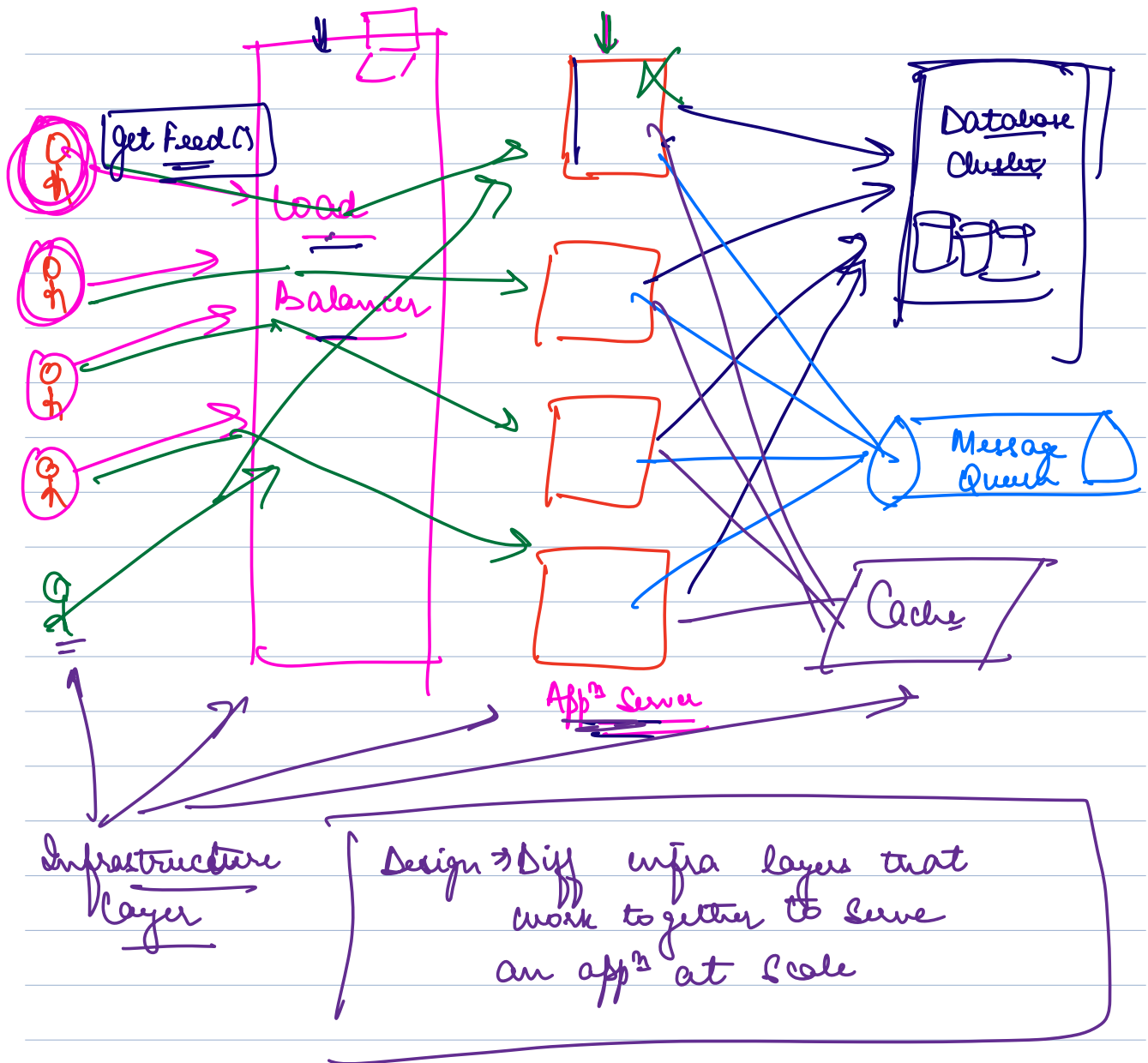
Intro to LLD And Intro to OOD

OOD

- ① Intro to LLD (Object Oriented Design)
- ② Why LLD is important
 - ↳ Interviews
 - ↳ Jobs.
- ③ ~~Structure of LLD module.~~
- ④ Intro to OOD

INTRO TO LLD





→ at the end what a comp works as is decided by the softw are that runs on the comp

LLD

↳ Details of code
↳ Implementation of the system

⇒ What are good ways to write code.

⇒ Helps you write better quality code.

Why LLD is important

1. JOBS

8 hr

1-2 hrs

≈ 12% of their day to day time actually writing code.

30 LPA

~~$\frac{88}{100} \times 30 \text{ LPA}$~~

↳ Look Good To Me

- Giving KT → Understand code → Coffee
- Meetings → Planning → Pair Prog
- Designing → Discuss Req → Scalable Alg^m
- Code Review → Understand code → T⁺ for scale-
- Debugging → Understand code
- Testing
- Arguing with PM → Requirements

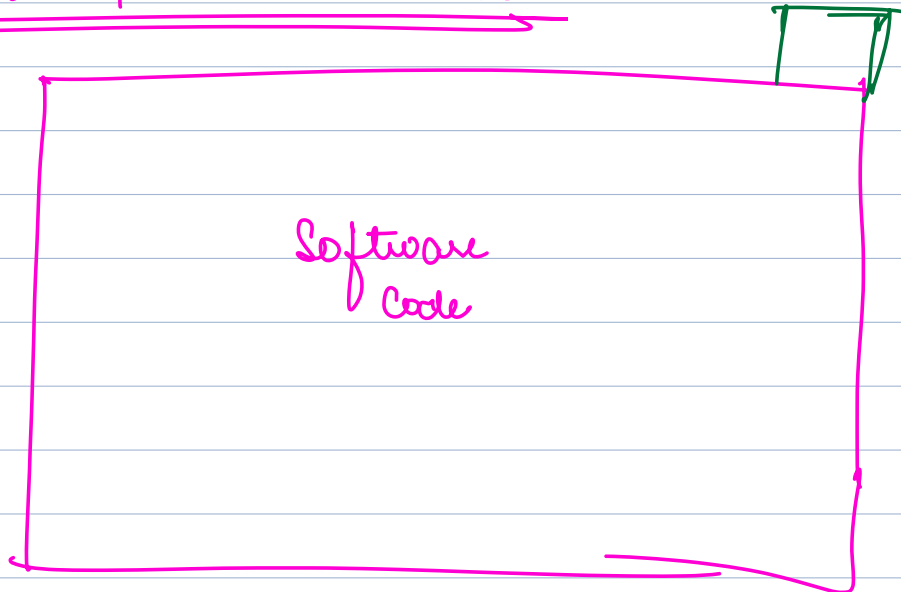
→ UD is going to make what a s/w eng does over 88% of their time easy

UD

- ① Readable / Understandable
- ② Req Gathering
- ③ Extensible
- ④ Maintainable.

Quality of output of work a SWE does.

Extensible vs Maintainable



Extensible → easy to add new features. (NEW REQ)

Maintainable → easy to keep running (NO NEW REQ)
→ Bugs
→ Updates to library

Mac OS 12
Mac OS 13

→ Platform Upgrades.

INTERVIEWS

→ every startup is going to have atleast one LLD round.

0-2 yoe → SDE1
2-5 yoe → SDE2
5-10 yoe → SDE3
10+ yoe → SDE4+

Type of Comp → Level ↓	Startups Facebook, Cred, Cricbuzz, Swiggy	MNCs Oracle, Adobe, Logipal, Viste	FAANG Apple, Google	Misc Amazon Apple
SDE1	1 Machine Coding Round	NONE	X	OOP
SDE2	11	LLD Round	X	LLD
SDE3+			X	11

Code + Design

Just Design

Only focus on LLD and OOP

Structure of LLD Model

⇒ 2 Months

- Theory
- ① OOP ⇒ 4 classes.
 - ② COLP Design Principles ⇒ 2 classes.
 - ③ Design Patterns ⇒ 5 classes
≈ 10 design pattern
 - Singleton
 - Factory
 - Builder
 - Observer
 - Prototype
 - Adapter
 - Strategy
 - Decorator
 - Facade
 - Flyweight
 - ④ UML Diag ⇒ (1 class)

⑤ Interview Practice

- ⇒ 2 classes design problem
- a.) How to approach design problems
 - b.) Design TTT
 - c.) Design Parking lot
 - d.) Book My Show
 - Splitwise
 - Cache
 - e.)
 - f.)

- ⇒ API Design
- ⇒ Schema Desi.
- ⇒ MVC
- ⇒ ORM
- ⇒ Spring Boot

In class ⇒ Java

concept ⇒ Python / Django
⇒ JS / TS / ExpressJS

INTRO TO OOP

Programming Paradigm

way to do something

↳ way to program

types of programming paradigm ↳ way to structure code / write code.

- Procedural Programming → C, Pascal, JS
- Object Oriented Programming → Python, Java
- Functional Programming → Java, JS
- Reactive Programming → Java, Rx-Java

Procedural Programming

↳ Old school name of function
↳ Set of instructions to get something done

↳ We structure our code into a bunch of procedures.

→ Each procedure may internally call other procedures

→ Execution of program starts from a special procedure → main()

```
acs {  
    cc }  
}
```

```
ccs {  
    ac }  
}
```

```
cc {  
}
```

```
main { }
```

```
cc }
```

```
cc }
```

```
}
```

problems with procedural programming

Problems with Procedural Prog

- We are learning C.
- Naman is teaching C.
- I am good.
- Achin is sending chat messages

⇒ Subject + Verbs

Something . Someone.

```

Print Student (String name, int age, String gender) {
    cout << name;
    cout << age;
    cout << gender;
    name = " ";
}

```

no ^m in ~~Access~~

```

Print Student (Student s) {
    cout << s.name;
    cout << s.age;
    cout << s.gender;
}

```

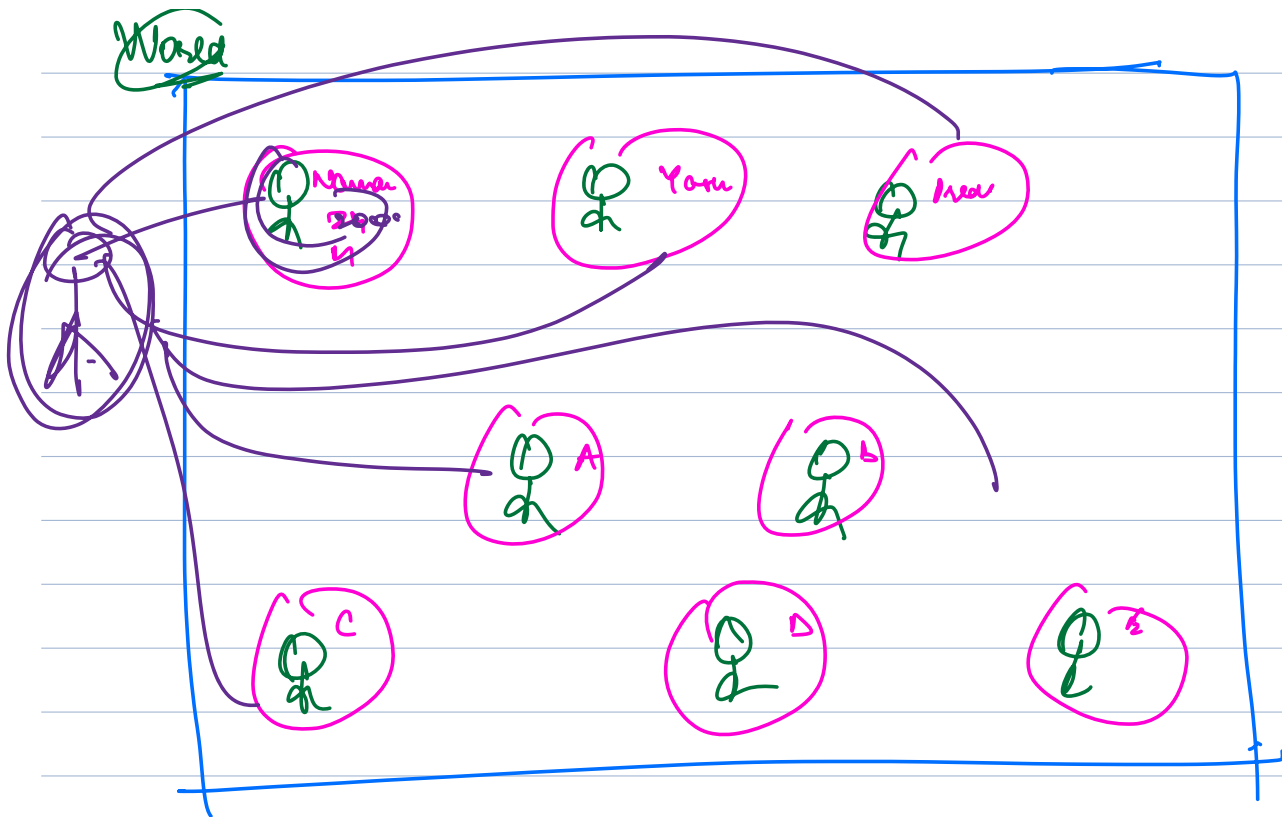
```

struct Student {
    String name;
    int age;
    String gender;
}

```

→ has access to every attr of Student
→ it can modify any attr of Student

IRL ⇒ Someone is doing something
Procedural ⇒ something is happening on someone.



Univex $\rightarrow$ 10000 s of procedure

Student {

XYZ.C
dance Student (Clove

$\rightarrow$

}

print Student (Student)

Student.print()

⇒ Action is happening on entity

⇒ Entity is performing action

⇒ Entities have no control on what happens to them.

OOP ⇒ Way to program where software system is structured around entity.

⇒ Entities are the core construct

↳ Actions are associated

↳ Control what they want to do

⇒ In proced prog it is more diff to understand what and why is happening in a S/W system.

⇒ In OOP easy to understand.

```

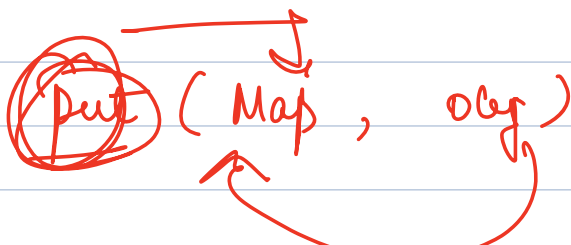
class Student {
    int age;
    String gender;
    String name;

    void print() {
    }

    void updateAge() {
    }
}

```

OOP $\Rightarrow$ becomes slow
 $\Rightarrow$ Memory Overhead



 Map.put(obj)

C

1 work $\Rightarrow$ 2 days

Java

$\Rightarrow$ 1 day

=

OOP

$\rightarrow$ entities are core construct in OOP

$\rightarrow$ every entity has attr and behaviour that they perform

Functional Prog $\Rightarrow$ Functions can be considered as entities



doSomething(f() { })

Head First Design Patterns